

Project 08

GitHub Execution & Workflow Plan (v1.0)



Lead Architect & Engineering Lead

AI Mock Interview & Coding Platform — 10-Member Sprint

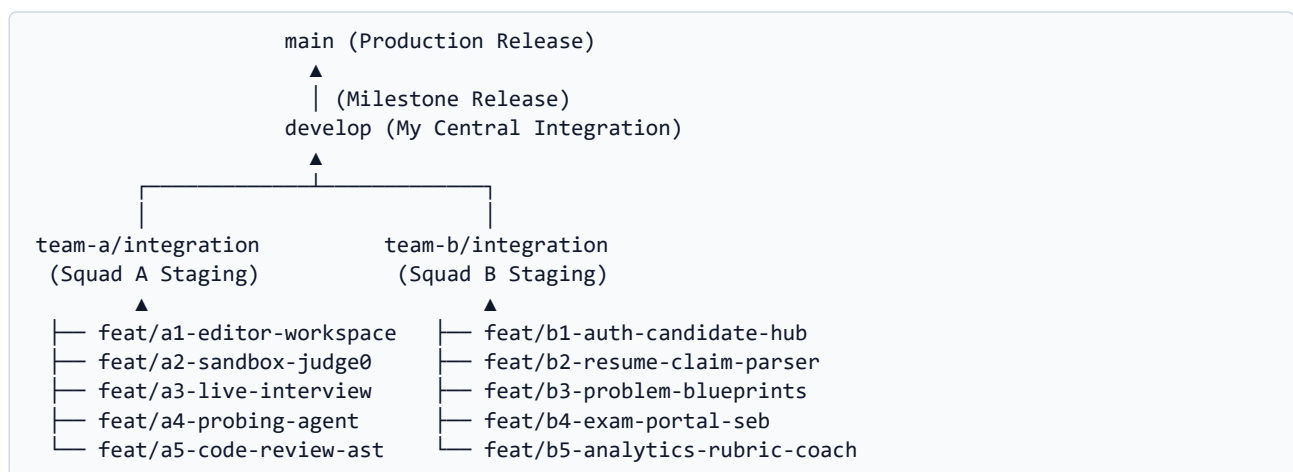
Daily Git Commands & Integration Protocol

1. Our Goal & How We Use This Plan

Our Project Goal: Build and integrate Project 08 (AI Mock Interview & Coding Platform) with our 10-member team across a structured 25-day sprint.

How We Work: To eliminate merge conflicts and ensure everyone has demonstrable viva evidence, each member owns an isolated vertical slice. Members push code only to their assigned files, open PRs to their squad integration branch, and I handle the central integration into `develop`.

2. Branching Hierarchy



3. 10-Member File Allocation Matrix

Squad A — Live Engine & Real-Time Coding

ID	Branch	UI Component	Backend API	DB Model	Test File
A1	feat/a1-editor-workspace	MonacoEditor.tsx	api/drafts.py	models/draft.py	test_drafts.py
A2	feat/a2-sandbox-judge0	TestConsole.tsx	api/execution.py	models/submission.py	test_execution.py
A3	feat/a3-live-interview	LiveCockpit.tsx	api/sessions.py	models/session.py	test_sessions.py
A4	feat/a4-probing-agent	ProbingStatus.tsx	api/probing.py	models/agent_turn.py	test_probing.py
A5	feat/a5-code-review-ast	CodeReviewCard.tsx	api/code_review.py	models/review.py	test_code_review.py

Squad B — Platform, Knowledge & Formal Exams

ID	Branch	UI Component	Backend API	DB Model	Test File
B1	feat/b1-auth-candidate-hub	AuthModal.tsx	api/auth.py	models/user.py	test_auth.py
B2	feat/b2-resume-claim-parser	ResumeViewer.tsx	api/resumes.py	models/resume_claim.py	test_resumes.py
B3	feat/b3-problem-blueprints	QuestionBank.tsx	api/questions.py	models/blueprint.py	test_questions.py

B4	feat/b4-exam-portal-seb	ExamPortal.tsx	api/exams.py	models/ formal_exam.py	test_exams.py
B5	feat/b5-analytics-rubric-coach	ScorecardView.tsx	api/ evaluations.py	models/rubric.py	test_evaluations.py

4. One-Time Setup (Day 1 Only)

Every member runs this once after I share the repo link:

```
# 1. Clone the repository
git clone https://github.com/<org>/project-08.git
cd project-08

# 2. Check out YOUR assigned feature branch (example: member A1)
git checkout feat/a1-editor-workspace

# 3. Verify you're on the correct branch
git branch
# * feat/a1-editor-workspace

# 4. Install dependencies
pip install -r requirements.txt      # backend
npm install                          # frontend (from frontend/ folder)
```

Rule: You only ever work on YOUR branch. Never check out someone else's branch or push to `develop` / `main` directly.

5. Daily Git Workflow — Full Command Reference

Follow this exact sequence every working day. I'm sharing this so there's zero confusion.

5.1. Step 1: Start of Day — Sync Your Branch

Before writing any code, pull the latest changes so you're not working on stale code:

```
# Make sure you're on YOUR branch
git checkout feat/<your-branch>

# Pull any changes I may have pushed to your branch (review feedback, fixes)
git pull origin feat/<your-branch>

# Sync with the latest squad integration branch to stay current
# Squad A members:
git pull origin team-a/integration --no-rebase

# Squad B members:
git pull origin team-b/integration --no-rebase
```

If you see a merge conflict at this step — **STOP**. Message me on the group. Do not try to resolve it yourself.

5.2. Step 2: Work on Your Assigned Files Only

Code your slice. You should only be editing YOUR assigned files:

```
Example for A1:
frontend/components/team_a/MonacoEditor.tsx
backend/api/drafts.py
```

```
backend/models/draft.py
backend/tests/test_drafts.py
```

Run your tests locally before committing:

```
# Run your specific test file
pytest backend/tests/test_drafts.py -v

# Or run all tests to make sure nothing else broke
pytest backend/tests/ -v
```

5.3. Step 3: Stage ONLY Your Files — Never Use `git add .`

```
# Check what you changed
git status

# Stage your specific files one by one
git add frontend/components/team_a/MonacoEditor.tsx
git add backend/api/drafts.py
git add backend/models/draft.py
git add backend/tests/test_drafts.py
```

NEVER run: `git add .` or `git add -A` — these will pick up files outside your slice and cause merge conflicts across the entire team.

5.4. Step 4: Commit with a Proper Message

Use this format — `feat(<module>): <what you did>`:

```
git commit -m "feat(drafts): add auto-save endpoint with 5s debounce"
git commit -m "feat(drafts): create Draft SQLAlchemy model with timestamps"
git commit -m "fix(drafts): handle empty code body in save request"
git commit -m "test(drafts): add unit tests for POST /api/code/drafts"
```

Tip: Commit often. Small commits are better than one giant commit at the end of the day.

5.5. Step 5: Push to Your Remote Branch

```
git push origin feat/<your-branch>
```

If this is your first push, use:

```
git push -u origin feat/<your-branch>
```

5.6. Step 6: Open a Pull Request (When Your Slice is Ready)

Go to GitHub and create a PR:

PR Title	feat(a1): Monaco Editor workspace with auto-save
PR Target	Squad A: team-a/integration Squad B: team-b/integration
PR Body	What you built (1–2 lines) + screenshot if UI + test output

Or from terminal using GitHub CLI:

```
# Squad A example:
gh pr create --base team-a/integration \
  --title "feat(a1): Monaco Editor workspace with auto-save" \
  --body "Implements MonacoEditor.tsx + POST /api/code/drafts + Draft model.
        All tests passing."

# Squad B example:
gh pr create --base team-b/integration \
  --title "feat(b1): Auth system with JWT and RBAC" \
  --body "Implements AuthModal.tsx + POST /api/auth/login + User model.
        All tests passing."
```

After PR is open: I review the code, leave comments if needed, and merge it into the squad integration branch. You do NOT merge your own PR.

6. If I Ask for Changes on Your PR

```
# 1. Make sure you're on your branch
git checkout feat/<your-branch>

# 2. Pull latest (in case I pushed a fix)
git pull origin feat/<your-branch>

# 3. Make the requested changes in your files

# 4. Stage, commit, push – same process
git add <your-changed-files>
git commit -m "fix(<module>): address review – <what you fixed>"
git push origin feat/<your-branch>

# The PR updates automatically. No need to open a new one.
```

7. Rules — Read This Twice

Rule	Why
Never push to develop or main	Only I merge into these branches
Never use git add . or git add -A	Picks up files outside your slice and causes conflicts

Never check out someone else's feature branch	Your work stays isolated on your branch
Never force push (<code>git push --force</code>)	Destroys history; message me if you're stuck
Never resolve merge conflicts alone	Tell me on the group — I'll handle it
Commit messages follow <code>feat/fix/test(<module>): format</code>	Keeps our git log clean and traceable
Open PR to squad integration branch, not <code>develop</code>	<code>team-a/integration</code> or <code>team-b/integration</code> only

8. My Integration Flow (Lead Only)

This is how I merge everything upward — team members don't need to do any of this:

```

feat/a1..a5 → PR → team-a/integration  (I review & merge)
feat/b1..b5 → PR → team-b/integration  (I review & merge)

team-a/integration ┌
                    │ → develop          (I merge both squads after testing)
team-b/integration ┘

develop → main      (Milestone releases only)
```

9. Quick Troubleshooting

Problem	What to Do
I committed to the wrong branch	Don't panic. Message me with the branch name and commit hash. I'll cherry-pick it.
<code>git push</code> says rejected	Run <code>git pull origin feat/<your-branch></code> first, then push again.
I see merge conflicts	Do NOT resolve them. Run <code>git merge --abort</code> and message me.
I don't see my branch	Run <code>git fetch --all</code> then <code>git checkout feat/<your-branch></code> .